

PHASE 5

SCALING

High-End System Design Interview Notes

THE BIG IDEA

Scaling means increasing a system's capacity so it can handle more traffic, data, or work while meeting its performance and reliability goals.

Vertical = Scale Up • Horizontal = Scale Out • Autoscaling = Adjust Capacity

Phase 5 Learning Map

1. What scaling means
2. Vertical scaling
3. Horizontal scaling
4. Stateless architecture
5. Stateful architecture
6. Autoscaling
7. Scaling trade-offs
8. Scaling a real application
9. Common scaling bottlenecks
10. Interview decision framework

MEMORY

Vertical = scale up. Horizontal = scale out. Real systems can use both.

1. 🚀 What Is Scaling?

Scaling is the ability of a system to handle increasing workload by adding resources or changing the architecture. Workload can mean users, requests per second, data size, background jobs, or concurrent connections.

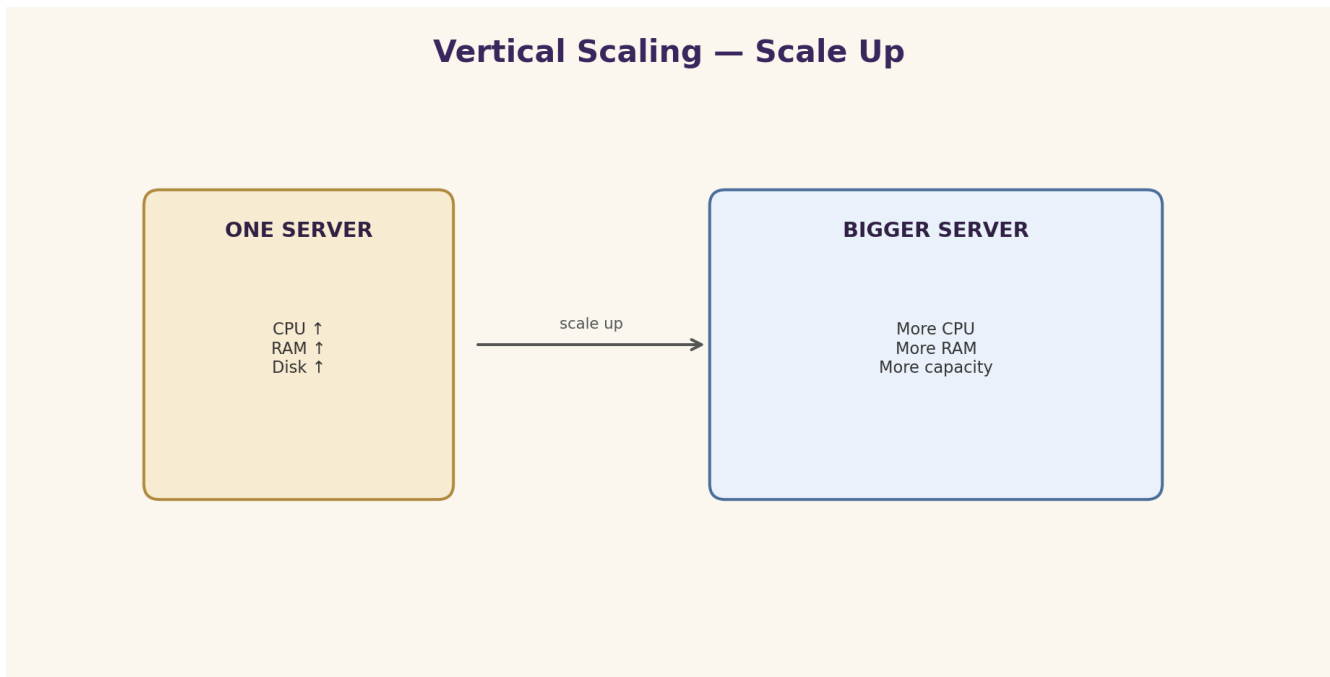
- Capacity — how much work the system can handle.
- Latency — how quickly requests complete.
- Throughput — how much work is completed per unit time.
- Availability — whether the service remains usable during failures or traffic increases.
- Cost efficiency — how much infrastructure is needed for the required capacity.

SYSTEM DESIGN VIEW

Do not scale just because traffic is high. First identify the bottleneck: CPU, memory, database, network, disk, locks, external APIs, or application design.

2. 📦 Vertical Scaling — Scale Up

Vertical scaling means making one machine or instance more powerful.



- Increase CPU.
- Increase RAM.
- Use faster storage or more I/O capacity.
- Move to a larger VM/instance size.

Typical use

- Simple applications with one or a few servers.

- Some database workloads where a larger machine is practical.
- Early-stage systems where architectural simplicity matters.

Advantages

- Simple architecture.
- Usually little application-level change.
- Useful when workload fits comfortably on a larger machine.

Limitations

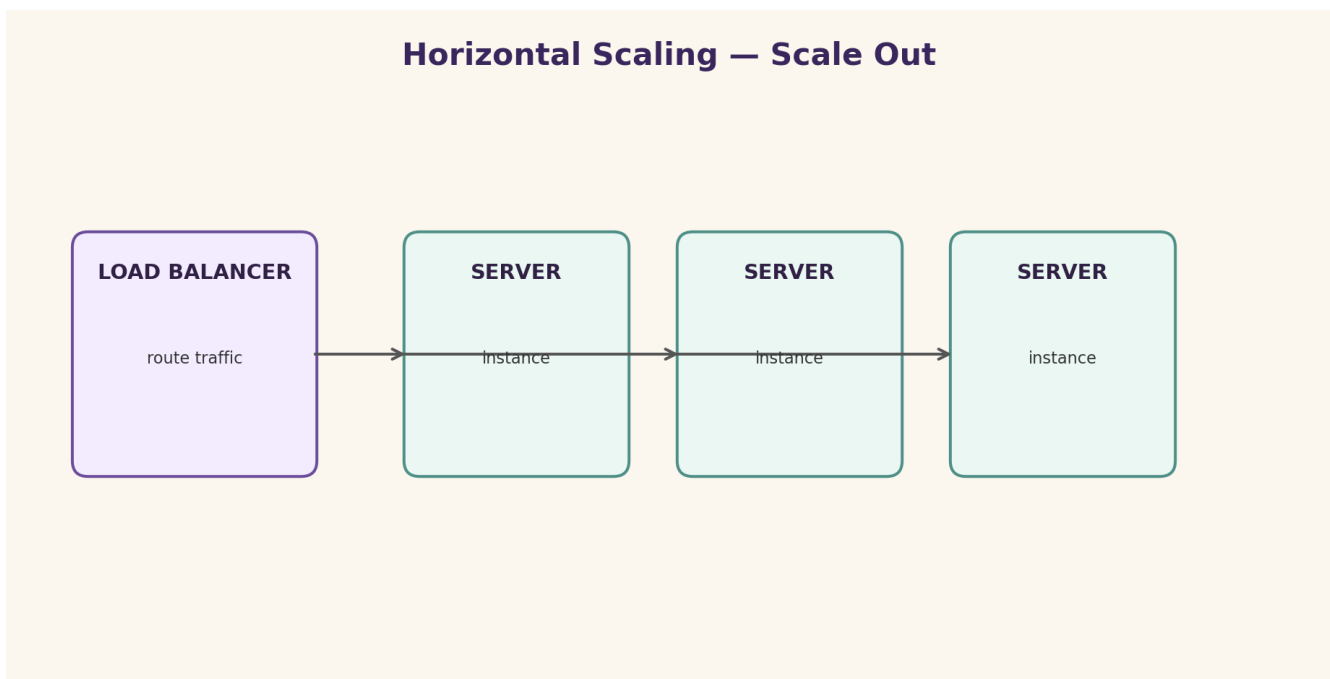
- Hardware/instance size has an upper limit.
- Larger machines can become expensive.
- A single machine can remain a failure point.
- Scaling up does not automatically solve every bottleneck.

INTERVIEW PHRASE

“Vertical scaling is simple, but it has a ceiling. It is often useful early or for workloads that benefit from a large machine, especially some database workloads.”

3. ➡ Horizontal Scaling — Scale Out

Horizontal scaling means adding more machines or application instances and distributing work across them.



- Add more application servers.
- Distribute incoming traffic using a load balancer.
- Remove instances when they are no longer needed.
- Allows capacity to grow across multiple machines.

Typical use

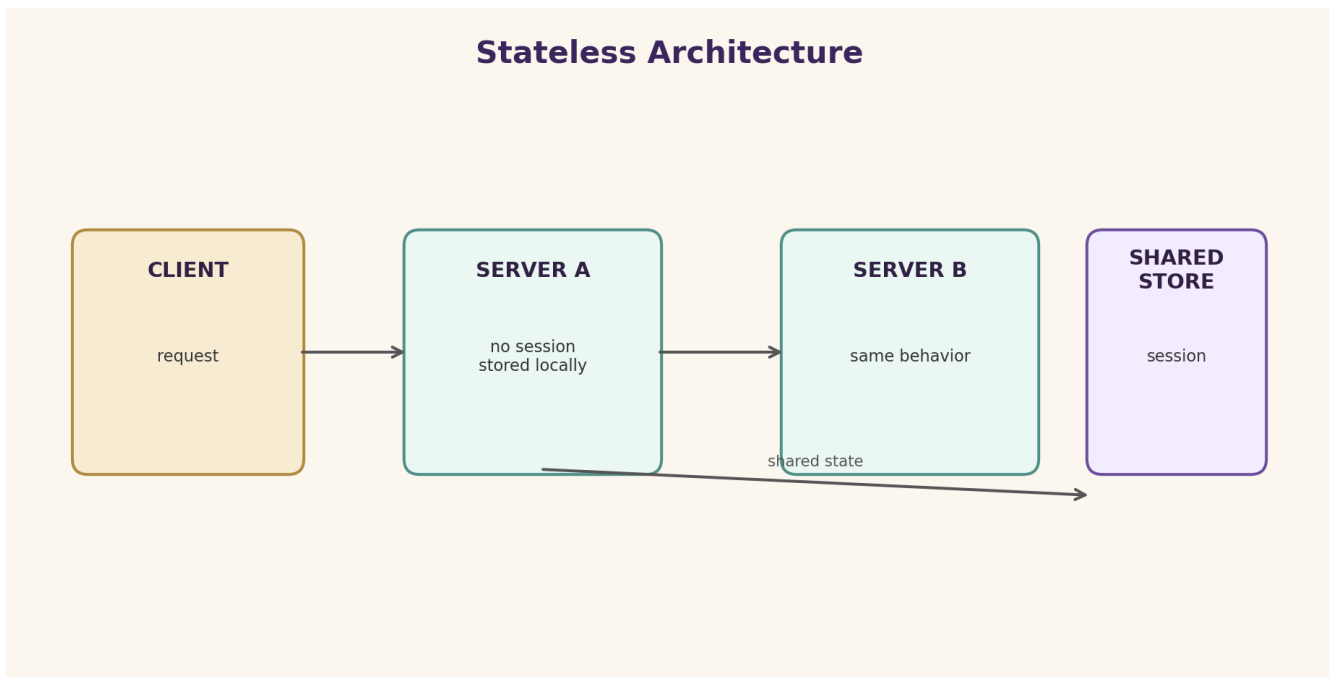
- High-traffic web applications.
- Distributed systems.
- Cloud-native applications.
- Services that need redundancy and multiple instances.

KEY IDEA

Horizontal scaling is not simply “buy more servers.” The application and its dependencies must be designed so multiple instances can work correctly together.

4. 🧩 Stateless Architecture

A stateless server does not keep a user's important session state only inside that server's local memory or disk between requests.



- Any healthy application instance can handle the next request.
- Session/state can be stored in shared systems such as a database or distributed cache when appropriate.
- Load balancers have more freedom to route requests across instances.
- Adding or replacing application instances becomes easier.

EXAMPLE

If Server A stores the user's login session only in its RAM, a later request sent to Server B may not know the session. Moving the session to a shared store can make the application layer stateless.

5. Stateful Architecture

A stateful server retains information about clients or ongoing work locally or within a tightly coupled server context.

- The next request may need to reach the same server or state holder.
- Examples can include some real-time connections, specialized session handling, and workloads where local state is intentionally important.
- Scaling becomes more complicated because state must be coordinated, replicated, moved, or routed correctly.

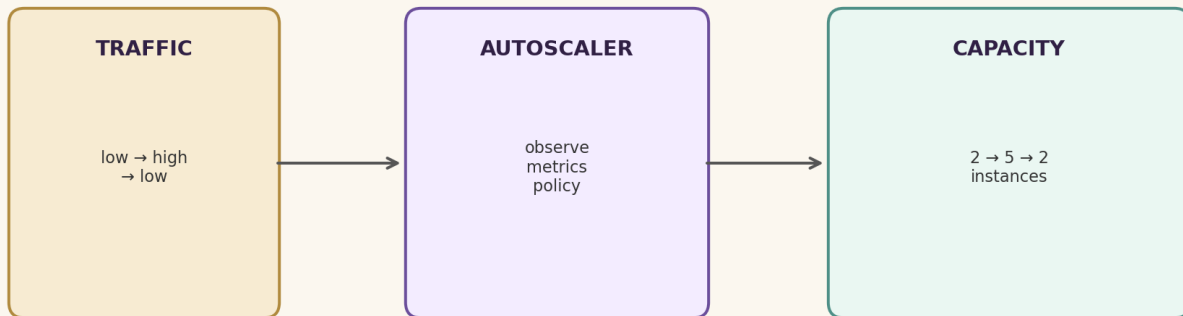
Stateful vs Stateless

Aspect	Stateless	Stateful
Client state	Stored outside individual server	Can remain tied to server/session
Scaling	Add/remove instances more easily	Requires state coordination
Routing	Flexible	May require affinity/sticky routing
Failure	Another instance can often continue	Local state may need recovery or migration

6. Autoscaling

Autoscaling automatically adjusts the number or size of resources based on demand and defined policies.

Autoscaling Based on Demand



- Scale out when traffic or resource usage rises.
- Scale in when demand falls, reducing unnecessary capacity.
- Policies can use CPU, memory, request rate, queue depth, latency, or other metrics.
- Usually works best when instances are replaceable and the application is designed for distributed operation.

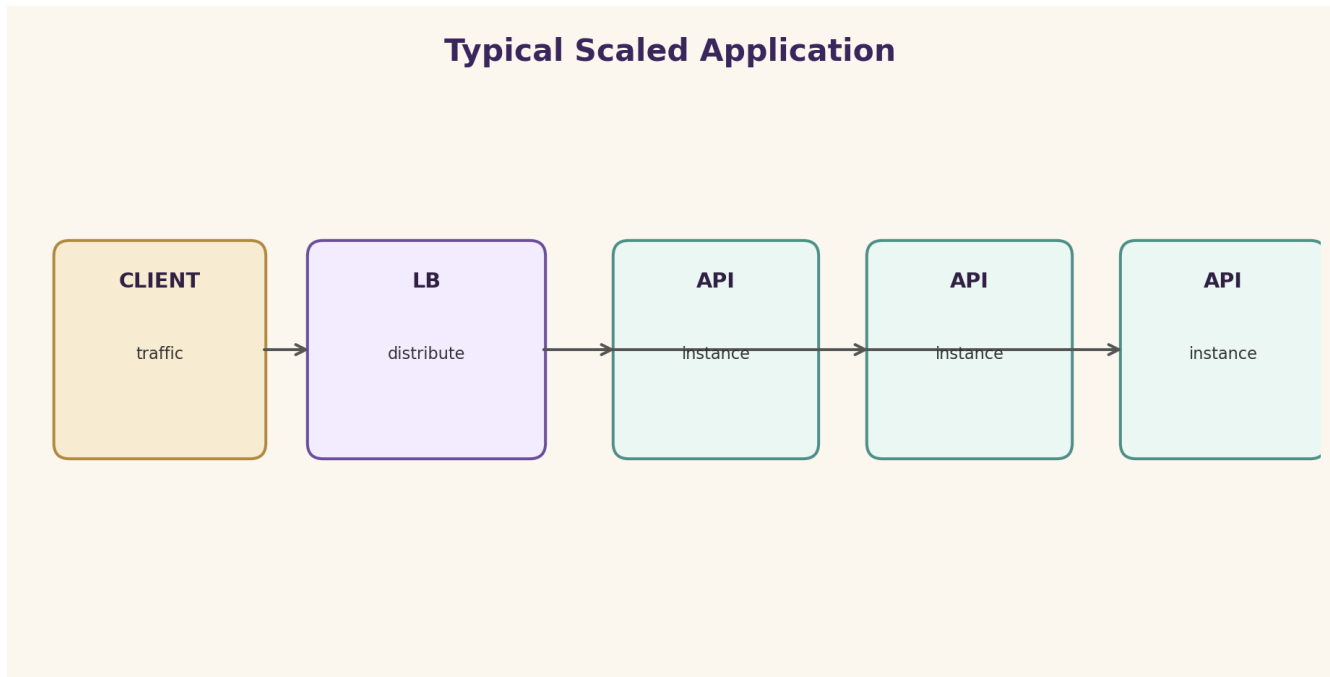
VARIABLE TRAFFIC

Autoscaling is especially useful when traffic changes significantly over time—for example, a service that has quiet periods and large traffic spikes.

7. Scaling Trade-offs

Approach	Benefits	Trade-offs
Vertical	Simple; minimal app changes	Capacity ceiling; bigger instances can be costly
Horizontal	Large capacity; redundancy; distributed growth	More architecture and operational complexity
Stateless	Easy instance replacement and routing	State must live elsewhere when needed
Stateful	Can simplify certain specialized workloads	Harder routing, failover, and scaling
Autoscaling	Matches capacity to demand	Needs good metrics, limits, warm-up behavior, and safe

8. 🏠 Scaling a Real Application



11. Client sends traffic.
12. Load balancer distributes requests.
13. Multiple stateless application instances process requests.
14. Shared cache/database stores important state and data.
15. Autoscaling adds or removes application instances as demand changes.
16. Monitoring measures latency, errors, CPU, memory, throughput, and saturation.

IMPORTANT

Scaling the application tier does not automatically scale the database. The database, cache, queues, storage, and external dependencies can become the next bottleneck.

9. 🧱 Common Scaling Bottlenecks

- CPU saturation — requests require more computation than the instances can provide.
- Memory pressure — too much in-process data, large objects, or leaks.
- Database bottleneck — connection limits, slow queries, locks, or insufficient capacity.
- Network bandwidth — too much traffic between clients, services, or storage.
- Single shared dependency — one service becomes a choke point.
- Synchronous chains — one slow dependency makes the whole request slow.
- Stateful design — local state prevents easy distribution.

BOTTLENECK RULE

When scaling does not improve performance, ask: “What resource is saturated now?” Scaling one layer can simply move the bottleneck to another layer.

10. Scaling Strategies in Order

17. Measure the current system.
18. Find the actual bottleneck.
19. Optimize inefficient code or queries.
20. Add caching where repeated reads justify it.
21. Scale vertically when it is the simplest effective solution.
22. Make the application stateless when horizontal scaling is needed.
23. Scale horizontally behind a load balancer.
24. Add autoscaling for variable demand.
25. Scale dependent systems such as databases, queues, and caches as needed.
26. Continuously monitor and revise based on measured workload.

11. Interview Question: Vertical vs Horizontal Scaling

STRONG ANSWER

“Vertical scaling increases the capacity of one machine by giving it more CPU, memory, or faster resources. It is simple but has a hardware or instance-size ceiling. Horizontal scaling adds more machines and distributes traffic, which is useful for high-traffic distributed systems, but requires the application and its dependencies to work correctly across multiple instances. In practice, real systems often combine both approaches.”

Follow-up: Why Stateless?

“A stateless application server does not depend on locally stored client session state. That lets a load balancer send requests to different healthy instances and makes adding or removing instances much easier.”

12. Interview Questions You Should Master

27. Explain vertical scaling with a real example.
28. Explain horizontal scaling with a web application example.
29. Why is horizontal scaling harder than simply adding servers?
30. What does stateless mean?
31. Why does statelessness make horizontal scaling easier?
32. When would you intentionally keep state?
33. What is autoscaling?
34. Which metrics can trigger autoscaling?

35. Why can scaling the application servers fail to solve a database bottleneck?
36. What happens if one application instance fails in a stateless architecture?
37. How would you scale an API from 100 requests/second to 10,000 requests/second?
38. What bottlenecks would you check before adding more servers?

13. One-Page Cheat Sheet

Concept	Remember
Vertical	Scale UP → bigger machine
Horizontal	Scale OUT → more machines
Stateless	Server does not keep required client session locally
Stateful	Server retains/ties important state to a server/session
Load Balancer	Distributes traffic across instances
Autoscaling	Automatically adjusts capacity based on demand
Simple systems	Vertical scaling can be sufficient
High traffic	Horizontal scaling is often useful
Variable traffic	Autoscaling can adjust capacity
Real systems	Can combine vertical + horizontal scaling
Core memory	Vertical = up; Horizontal = out

14. Mini Design Exercise

Imagine a college attendance API suddenly receives 50× its normal traffic when thousands of students open the app at the same time.

39. Identify the likely bottleneck using metrics.
40. Put a load balancer in front of multiple application instances.
41. Keep application instances stateless where practical.
42. Use a shared cache for frequently repeated read data.
43. Protect the database from excessive repeated queries.
44. Configure autoscaling for traffic/resource spikes.
45. Monitor latency, error rate, database connections, CPU, memory, and saturation.
46. Test scale-in and scale-out behavior before relying on it in production.

MASTER QUESTION

Do not start with “How many servers?” Start with “What workload are we handling, what is the bottleneck, and what capacity target are we trying to meet?”

PHASE 5 COMPLETE ✓

Vertical = Scale Up • Horizontal = Scale Out • Stateless = Easier Distribution